QA AUTOMATION & TEST ENGINEERING PORTFOLIO

Enterprise Portal Test Automation Framework

A resilient, layered Keyword-Driven E2E test automation architecture built with Robot Framework, Selenium WebDriver, and REST API integration with Jenkins CI/CD pipeline automation.

AUTHOR & ENGINEER

Kharis Ediwardana

CORE TECHNOLOGIES

Robot Framework, Selenium, Python

DESIGN PATTERN

Layered Keyword-Driven / Page Object

CI/CD PIPELINE

Jenkins (Declarative & Robot Plugin)

GITHUB REPOSITORY

github.com/kharisediwardana/enterprise-portal-automation-robotframework

1. Executive Summary & Project Overview

Modern enterprise systems often bridge complex public-facing citizen self-service portals with rigorous back-office case management platforms. Ensuring reliability across high-stakes digital submissions (such as dispute appeals, legal notices, and multi-format document verification) requires an automated regression harness that is resilient, maintainable, and seamlessly integrated into a continuous delivery pipeline.

100+	20+	4	100%
AUTOMATED TEST CASES	FILE UPLOAD FORMATS TESTED	DYNAMIC WAIT TIERS	DECOUPLED LOGIC LAYERS

Technical Scope & Challenges

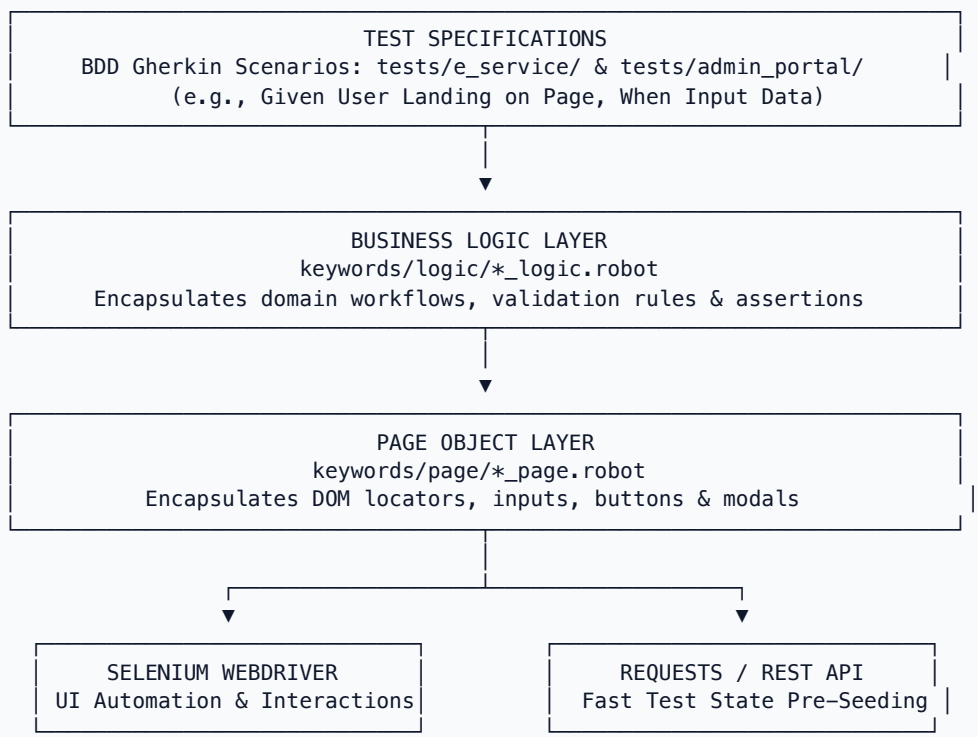
- **Asynchronous UI Complexity:** Heavy modal dialogues, role-based conditional inputs, and multi-step file upload pipelines frequently triggered timing issues in naive automation scripts.
- **Multi-Format Ingestion:** Portal forms required validating comprehensive MIME types, boundary limits, and batch CSV processing for case notices.
- **Multi-Environment Drift:** Deployments moved across multiple staging environments (SIT and Staging), requiring zero-downtime test configuration shifts without code modifications.

Automated Architecture Solution

An end-to-end automated testing suite was engineered leveraging **Robot Framework**, **Selenium WebDriver**, **RequestsLibrary**, and **FakerLibrary**. Integrated with **Jenkins CI/CD**, the suite automatically executes on code changes or scheduled nightly builds, publishing pass/fail trends and failure screenshots directly to the Jenkins job dashboard.

2. Framework Architecture & Design Pattern

The framework strictly adheres to a **Layered Keyword-Driven Architecture** combined with the **Page Object Model (POM)**. This guarantees that UI updates (such as CSS selector modifications) are localized to a single file without breaking upstream business test cases.



keywords/page/ (Page Objects)

Houses concrete web element locators (ID, XPath, CSS) and primitive browser actions. Isolates HTML structure changes from test scripts.

keywords/logic/ (Business Logic)

Orchestrates reusable business steps (e.g., logging in as a specific role, submitting multi-page appeals, checking audit trails).

tests/ (Executable Scenarios)

Declarative test cases written using business terminology and tagging (`Smoke`, `Regression`, `PLUS12`).

helper.robot (Cross-Cutting Utilities)

Viewport scrolling scripts, dynamic mock data generators (FakerLibrary), and dynamic wait wrappers.

3. Core Engineering Highlights

1. Resilient Dynamic Wait Strategy (Eliminating Flakiness)

Arbitrary, hardcoded `Sleep` statements are the primary culprit behind test flakiness and bloated CI/CD runtimes. This framework implements a 4-tier conditional dynamic wait hierarchy:

Wait Tier	Threshold	Purpose & Application
TURBO_TIMEOUT	1s	Micro-waits for inline CSS animations, DOM transitions, and dropdown expansions.
SHORT_TIMEOUT	3s	Standard input element visibility, button enabling, and toast notification checks.
MEDIUM_TIMEOUT	5s	Modal dialogue appearances, async data table row populations, and tab switches.
LONG_TIMEOUT	10s	Heavy payload ingestion, server-side PDF generation, and batch export downloads.

2. Multi-Format File Attachment Verification Matrix

Enterprise citizen portals must strictly regulate user attachments. The test suite includes an automated matrix in `Media/` to test boundary conditions across 20+ file formats:

- **Valid Formats:** `.pdf`, `.docx`, `.xlsx`, `.csv`, `.png`, `.jpg`, `.zip`.
- **Negative & Boundary Testing:** Unsupported extensions (`.zippp`, `.exe`), empty payloads (`Empty.csv`), corrupted records (`invalidNotice.csv`), and oversized media files.
- **Automated Batch Processing:** Testing automated CSV ingestion with dynamic parsing of multiple notice records in a single batch pass.

3. Hybrid UI & REST API Automated State Provisioning

Testing deep UI workflows often requires an entity to exist in a specific lifecycle state (e.g., an unpaid notice or escalated dispute). Rather than performing repetitive UI steps, the framework uses **RequestsLibrary** (`tests/api/api_test.robot`) to seed database records in milliseconds:

- `POST /sessions`: Immediately provisions an active parking/case session.
- `PATCH /records/{id}`: Modifies timestamp and status flags to trigger automated appeal eligibility rules.
- `GET /sync-data/pull-case`: Triggers backend microservice sync before initiating UI verification.

4. Multi-Environment Isolation & Zero Hardcoding

Configuration is externalized into dedicated files (`config/environment_sit.robot` and `config/environment_staging.robot`). Test suites dynamically load endpoints and credentials via CLI variables without altering source code.

4. Code Architecture & Implementation Walkthrough

Declarative BDD Test Case Specification

```
*** Settings ***
Documentation    Citizen Portal Appeal Submission & Dispute Management
Resource         ../../resources.robot
Suite Setup      Open Browser And Go To Public Portal
Test Setup       Navigate To Public Portal
Test Teardown    Capture Page If Test Failed
Suite Teardown   Close All Browsers
Force Tags       AppealPortal

*** Test Cases ***
PLUS_12.1_001 User Creates New Dispute Case With Valid Particulars
    [Tags]    Smoke    Regression
    Given User Landing on Public Appeal Page
    When User Accepts Terms and Declaration
    And User Inputs Appeal Notice Particulars       ${TEST_PO_NUM}
    And User Uploads Supporting Evidence Document    ${SUPPORT_FILE_PDF}
    Then User Able Submit Create New Case Data
    And System Displays Successful Acknowledgment Number
```

Business Logic Encapsulation (`keywords/logic/`)

```
*** Keywords ***
User Inputs Appeal Notice Particulars
    [Arguments]    ${notice_number}
    Wait And Input Text       ${INPUT_NOTICE_FIELD}    ${notice_number}    timeout=${SHORT_TIMEOUT}
    Click Element             ${BTN_VERIFY_NOTICE}
    Wait Until Element Is Visible    ${ALERT_VALID_RECORD}    timeout=${MEDIUM_TIMEOUT}

User Uploads Supporting Evidence Document
    [Arguments]    ${file_path}
    # Resilient file upload without relying on native OS dialogue
    Scroll Element Into View  ${UPLOAD_CONTAINER}
    Choose File               ${INPUT_FILE_UPLOAD}    ${file_path}
    Wait Until Page Contains Element    xpath=//*[contains(text(),\"Upload Successful\")]
```

Automated Teardown & Failure Diagnostics (`tests/TestTeardown.robot`)

```
*** Keywords ***
Capture Page If Test Failed
    [Documentation]    Captures diagnostic screenshots directly into output directory upon failure
    Set Screenshot Directory    ${OUTPUT_DIR}
    Run Keyword If Test Failed  Capture Page Screenshot    filename=failure-{index}.png
    Delete All Cookies
```

5. Jenkins CI/CD Pipeline & Test Execution

Continuous Integration Workflow (Jenkins Declarative Pipeline)

The test suite is automated via a production-grade **Jenkins Declarative Pipeline (`Jenkinsfile`)** configured to provide continuous quality feedback:

- **Parameterized Builds:** Enables on-demand target environment selection (`SIT` or `STAGING`), headless mode toggling, and tag filtering (e.g. `Smoke`, `Regression`, `PLUS12`).
- **Robot Framework Jenkins Plugin:** Automatically parses `output.xml` and publishes native pass/fail trend graphs, duration analytics, and threshold evaluation (90% pass / 75% unstable) directly onto the Jenkins build dashboard.
- **Automated Diagnostic Archiving:** In the `post` block, execution artifacts (`log.html`, `report.html`, and timestamped failure screenshots) are permanently archived for fast root-cause triage.

Declarative Pipeline Definition (`Jenkinsfile`)

```
pipeline {
    agent any
    parameters {
        choice(name: 'ENVIRONMENT', choices: ['STAGING', 'SIT'], description: 'Target environment')
        string(name: 'TEST_TAGS', defaultValue: 'Smoke', description: 'Filter (Smoke, PLUS12)')
        booleanParam(name: 'HEADLESS', defaultValue: true, description: 'Headless mode')
    }
    stages {
        stage('Execute Robot Framework Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    robot --variable HEADLESS:${HEADLESS} \\
                          --variablefile config/environment_${ENVIRONMENT.toLowerCase()}.robot \\
                          --include ${TEST_TAGS} --outputdir results tests/ || true
                '''
            }
        }
    }
    post {
        always {
            // Publish native test metrics using Robot Framework Jenkins Plugin
            step([$class: 'RobotPublisher', outputPath: 'results', passThreshold: 90.0, unstableThreshold: 75.0])
            archiveArtifacts artifacts: 'results/*.png, results/*.html, results/*.xml', allowEmptyArchive: true
        }
    }
}
```

Core Competencies Demonstrated

🎯 Test Automation Architecture

Modular layered frameworks with Robot Framework, Python & POM.

⚡ Flakiness Mitigation

Tiered dynamic condition waits, viewport scrolling & session teardown.

🔗 Hybrid API + UI Testing

Fast state pre-seeding via REST endpoints with RequestsLibrary.

⚙️ Jenkins CI/CD Automation

Declarative pipelines, parameterized runs & Robot Plugin trends.

Source Code & Test Artifacts Available on GitHub

Explore the complete implementation, keyword hierarchies, and Jenkinsfile:

https://github.com/kharisediwardana/enterprise-portal-automation-robotframework